

Modül 16

Performance

Optimization Patterns

Modül:	16 / 30
Ders Sayısı:	8 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

📖 Modül 16'ya Hoş Geldin

Performance — büyük uygulamaların ayakta kalmasının anahtarı. Modern Angular'da signal-based rendering, OnPush change detection, zoneless mode, bundle optimization. Hepsini öğreneceğiz.

Öğrenecekler:

- ✓ Change detection nasıl çalışır
- ✓ OnPush strategy kullanımı
- ✓ Signal-based change detection avantajları
- ✓ Zoneless Angular
- ✓ @defer ile lazy loading (recap)
- ✓ Bundle analizi ve optimization
- ✓ TrackBy / track for @for
- ✓ Image optimization (NgOptimizedImage)

Change Detection Mekaniği

Angular ne zaman ve neden re-render eder?

□ Default Change Detection

Klasik Angular (Zone.js ile): Async olay olunca (click, HTTP, timer) tüm tree taranır.

```
AsyncEvent (click, http, timer)
↓
Zone.js detects
↓
Angular runs Change Detection
↓
Tree traversal (root → leaf)
↓
Her component:
- Template expression'ları çalıştır
- Bindings'i karşılaştır
- DOM güncelle (gerekirse)
```

⚠ Problem: Gereksiz Re-render

```
@Component({
  template: `<p>{{ expensiveCalc() }}</p>`
})
export class MyComponent {
  expensiveCalc() {
    console.log('Recalculating...');
    return /* heavy work */;
  }
}
```

// Sayfada bir click olsa, bu method TEKRAR çalışır
// Her CD cycle'da çalışır – pahalı!

OnPush Change Detection Strategy

Component'in change detection'ını kontrol altına almak.

OnPush Aktivasyonu

```
import { ChangeDetectionStrategy } from '@angular/core';

@Component({
  selector: 'app-user-card',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `...`
})
export class UserCardComponent {
  user = input.required<User>();
}
```

OnPush Ne Zaman Re-render Eder?

- ✓ Input referansı değişince (object identity)
- ✓ Event tetiklenince (click, etc — component içinde)
- ✓ Signal değişince (modern)
- ✓ async pipe emit edince
- ✓ markForCheck() manuel çağrılınca

⚠ Anti-Pattern: Object Mutation

```
// ❌ OnPush ile çalışmaz
this.user.name = 'New Name'; // Referans aynı, OnPush tetiklenmez

// ✓ Yeni object ile
this.user = { ...this.user, name: 'New Name' };

// ✓ Signal ile
this.user.set({ ...this.user(), name: 'New Name' });
```

Signal-Based Change Detection

Modern Angular: signal'lar zaten OnPush gibi davranır.

⚡ Signal'ların Sihiri

Signal kullanan component otomatik OnPush gibi davranır. Sadece signal değişince tetiklenir, başka değişikliklerden etkilenmez.

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush, // Best practice
  template: `<p>{{ count() }}</p>`
})
export class CounterComponent {
  count = signal(0);
  // Sadece count değişince re-render olur
}
```

📦 Best Practice

📦 Önerim

Tüm component'lerini OnPush + Signal ile yaz. Performance'ın temel taşı.

Zoneless Angular

Zone.js'siz yaşam. Daha hızlı, daha küçük bundle.

⚙️ Zoneless Setup

```
// app.config.ts
import { provideExperimentalZonelessChangeDetection } from '@angular/core';

export const appConfig: ApplicationConfig = {
  providers: [
    provideExperimentalZonelessChangeDetection(),
    // ...
  ]
};

// polyfills.ts'den Zone.js'i kaldır
// import 'zone.js'; // ← bunu sil
```

📦 Zoneless'in Faydaları

- ✓ Bundle ~100KB küçülür (Zone.js kaldırılır)
- ✓ Performance artışı: gereksiz CD yok
- ✓ Debugging kolay: stack trace temiz
- ✓ SSR daha hızlı

⚠️ Şartlar

- ✓ Signal-only state — Observable'lar için async pipe
- ✓ RxJS Observable'lar için toSignal()
- ✓ 3rd party lib'lar zone-aware olmalı (çoğu sorunsuz)
- ✓ setTimeout/Promise içinde change detection trigger yok

@defer ile Lazy Loading

Modül 6'da gördüğümüz @defer'in performance impact'i.

❑ Bundle Splitting

```
<!-- @defer olmadan -->
<app-header />
<app-hero />
<app-comments />    <!-- Bundle'a girer -->
<app-footer />

<!-- @defer ile -->
<app-header />
<app-hero />
@defer (on viewport) {
  <app-comments />    <!-- Ayrı chunk, lazy load -->
}
<app-footer />
```

❑ Ölçülebilir Etki

Metric	Sans @defer	Avec @defer	İyileşme
Initial bundle	450 KB	280 KB	-38%
	1.8s	1.1s	-39%
	2.5s	1.5s	-40%
	3.2s	2.0s	-37%

Bundle Analizi

Bundle'ında ne var, ne kadar büyük?

□ source-map-explorer

```
# Install
npm install -D source-map-explorer

# Build with source maps
ng build --source-map

# Analyze
npx source-map-explorer dist/my-app/browser/main-*.js
```

□ Bundle Visualization

```
# webpack-bundle-analyzer
npm install -D webpack-bundle-analyzer

# Çalıştır
npx webpack-bundle-analyzer dist/my-app/browser/stats.json
```

□ Optimization Stratejileri

- ✓ Heavy imports: Lazy load (Chart.js, PDF libs)
- ✓ Tree-shakable kütüphane seç (lodash-es > lodash)
- ✓ Specific imports: `import { debounce } from 'lodash-es'`
- ✓ moment.js yerine date-fns veya native
- ✓ Polyfill'leri review et — modern target için gereksizler

track for @for — Performance Kritik

@for içinde track zorunlu. Yanlış kullanım yavaşlatır.

❑ Doğru Track

```
<!-- ID ile track – EN İYİ -->
@for (user of users(); track user.id) {
  <app-user [user]="user" />
}

<!-- Primitive list -->
@for (color of colors; track color) {
  <span>{{ color }}</span>
}
```

❑ Yanlış Track

```
<!-- ❑ Index kullanma (eğer liste yeniden sıralanıyorsa) -->
@for (user of users(); track $index) {
  <!-- Sıralama değişince Angular kafası karışır -->
}

<!-- ❑ Object referansı -->
@for (user of users(); track user) {
  <!-- Spread sonrası yeni referans → tüm DOM yeniden -->
}
```

❑ Neden Önemli?

Track olmadan Angular her item'ı silip yeniden oluşturur. ID ile track ederse, sadece değişeni günceller. 1000 item için %95+ performans farkı.

NgOptimizedImage

Modern image optimization Angular built-in.

□ Setup

```
import { NgOptimizedImage } from '@angular/common';

@Component({
  imports: [NgOptimizedImage],
  template: `
    <img
      ngSrc="hero.jpg"
      width="1200"
      height="600"
      priority>

    <img
      ngSrc="thumb.jpg"
      width="200"
      height="200"
      [ngSrcset]=" '200w, 400w' "
      sizes="(max-width: 768px) 100vw, 50vw"
      loading="lazy">
  `,
})
```

⚡ Faydaları

- ✓ Lazy loading default (sadece priority hariç)
- ✓ Async decoding — UI bloklamaz
- ✓ Preload priority image'ları
- ✓ Auto srcset ile responsive
- ✓ Warning — büyük image'lar için console uyarı

□ LCP Optimization

□ LCP = Largest Contentful Paint

Hero/header image'larına priority ekle. Browser ön-yükler, LCP iyileşir. Diğerlerine lazy.

Modül 16 Özeti

Performance konusunda artık tam donanımlısın. Modern Angular'ın signal-based render'ı, OnPush, Zoneless, lazy loading, image optimization — hepsi cebinde.

Neler Öğrendin?

- ✓ Change detection mekaniği
- ✓ OnPush strategy
- ✓ Signal-based CD (otomatik OnPush)
- ✓ Zoneless Angular
- ✓ @defer recap
- ✓ Bundle analizi
- ✓ track for @for
- ✓ NgOptimizedImage

Sıradaki Modül

Modül 17 — Testing. Unit test (Jest/Vitest), integration test, component testing best practices.